Quantum algorithms

Quantum computing

G. Chênevert

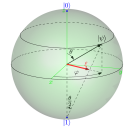
January 28, 2026





Last time

- Qubit states: $\alpha |0\rangle + \beta |1\rangle$ or (up to equivalence)



- Single qubit gates: X, Y, Z, H, $P(\theta)$, $U(\varphi, \theta, \lambda)$, ...
- Multiple qubit gates: SWAP, CX, Toffoli = CCX

Quantum gates on n qubits can be represented by $n \times n$ unitary matrices

$\Longrightarrow$ all operations (except measurements) are *reversible*

Reversible evaluation

In general: for $f : \{0,1\}^n \to \{0,1\}$ any boolean function

consider the gate U_f on $n+1$ qubits defined by

$$U_f : |\mathbf{x}\rangle \otimes |y\rangle \mapsto |\mathbf{x}\rangle \otimes |y \oplus f(\mathbf{x})\rangle.$$

You may check that this is a unitary operator on $\mathcal{V}_{2^n} \otimes \mathcal{V}_2 = (\mathcal{V}_2)^{\otimes(n+1)}$

that allows to evaluate f on $\mathbf{x}$ without losing $\mathbf{x}$:

$$U_f(|\mathbf{x}\rangle|0\rangle) = |x\rangle|f(\mathbf{x})\rangle.$$

Fact

For any boolean function $f : \{0,1\}^n \to \{0,1\}$

the quantum gate U_f can be implemented using only Toffoli and X gates.

Classical analogue: any boolean function can be generated using only the NAND gate

Exercise: come up with a circuit that implements the U_f gate for $f(x, y) = x \,\mathrm{OR}\, y$

Quantum algorithms

Quantum algorithms

Towards Shor

Period detection

Shor's algorithm

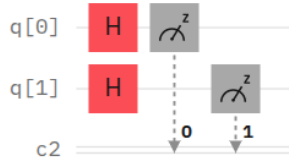
Shor IRL

Grover's algorithm

Grover IRL

Warm-up

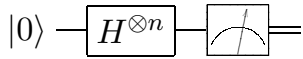
What is this circuit doing?



Answer: outputs a random integer uniformly chosen among $[\![0, 4[\![$.

True random number generator

In general



ouputs any integer in $[\![0, 2^n[\![$ with probability $\frac{1}{2^n}$.

$$\underbrace{H|0\rangle \otimes \cdots \otimes H|0\rangle}_{n} = \frac{1}{2^{n/2}} \sum_{x<2^n} |x\rangle$$

Faster computations

The whole point of quantum computing is that some quantum algorithms exhibit

quantum advantage: *i.e.* run "faster" than the best known classical algorithms

In extreme cases, this leads to

quantum supremacy: *i.e.* the ability to compute things that could never practically be achieved with classical computers.

2019: Sampling random quantum circuits on 53 qubits (Google) supremacy?

2022: Quantum computational advantage using photons (USTC)

Complexity of an algorithm

Classically: the complexity of an algorithm $\mathcal{A}$ is a bound on the number of computing steps needed for an input of a given size

i.e. the function

$$n \mapsto \max_{|x|=n} N_{\mathcal{A}}(x)$$

where $N_{\mathcal{A}}(x)$ denotes the number of steps used to perform $\mathcal{A}$ on x in a given computing model (usually: Turing machines)

Quantum computing model

There exist a (rather inconvenient) notion of quantum Turing machine

Most people prefer to use the (equivalent) **quantum circuit model**:

- given input x: a circuit $U_{\mathcal{A},x}$ made out of quantum gates taken from a standard generating set is prepared
- the output y is the result of the measure of $U_{\mathcal{A},x}|0\rangle$
- (then some classical post-processing may be applied)

The **complexity** of the quantum algorithm is a bound on the number of gates needed:

$$n \mapsto \max_{|x|=n} |U_{\mathcal{A},x}|.$$

Algorithms with quantum advantage

Two algorithms displaying a clear quantum advantage over the classical counterparts:

- **Grover**'s algorithm

  unstructured search among N items in $\mathcal{O}(\sqrt{N})$

- **Shor**'s algorithm

  factoring a n-bit integer in $\mathcal{O}(n^2 \log n)$

Quantum algorithms

Quantum algorithms

Towards Shor

Period detection

Shor's algorithm

Shor IRL

Grover's algorithm

Grover IRL

Quantum circuits

In the end, a quantum circuit is just a way to describe a big unitary matrix.

n qubits: $2^n \times 2^n$ unitary matrix

Things we can implement using unitary matrices:

- reflections
- rotations
- ...
- **Fourier transforms**

Recall: Discrete Fourier Transform

N-point Fourier transform of a sequence $x[0], \ldots, x[N-1]$:

$$y[k] = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} e^{-\frac{2\pi ijk}{N}} x[j]$$

Matrix formulation:

$$\mathbf{y} = \mathcal{F}\,\mathbf{x} \qquad \text{with} \qquad \mathcal{F} = \frac{1}{\sqrt{N}} \begin{bmatrix} 1 & 1 & 1 & \ldots & 1 \\ 1 & \zeta & \zeta^2 & \ldots & \zeta^{N-1} \\ \vdots & \vdots & & & \vdots \\ 1 & \zeta^{N-1} & \zeta^{2(N-1)} & \ldots & \zeta^{(N-1)(N-1)} \end{bmatrix}$$

where $\zeta = e^{-\frac{2\pi i}{N}}$ is *a* primitive N-th root of unity

Inverse Fourier Transform

If

$$y[k] = \frac{1}{\sqrt{N}} \sum_{j<N} \zeta^{jk} x[j]$$

then

$$x[j] = \frac{1}{\sqrt{N}} \sum_{k<N} \zeta^{-jk} y[k]$$

$$= \frac{1}{\sqrt{N}} \sum_{k<N} (\zeta^*)^{jk} y[k]$$

with matrix $\mathcal{F}^{-1} = \mathcal{F}^* = \mathcal{F}^\dagger$

Fourier transforms are unitary

Quantum Fourier Transform

Suppose we have a quantum state $|\psi\rangle \in \mathcal{V}_N$:

$$|\psi\rangle = \sum_{x<N} \alpha_x |x\rangle$$

Its **Fourier transform** is the state

$$\mathcal{F}|\psi\rangle = \sum_{y<N} \beta_y |y\rangle$$

defined by

$$\beta_y = \frac{1}{\sqrt{N}} \sum_{x<N} \zeta^{xy} \alpha_x.$$

Quantum Fourier Transform

In other words: from a theoretical point of view

QFT of a state = DFT of the probability amplitudes

Often written in the equivalent form:

$$\mathcal{F}\,|x\rangle = \frac{1}{\sqrt{N}} \sum_{y<N} \zeta^{xy}\,|y\rangle$$

Naive classical algorithm: $\mathcal{O}(N^2)$ operations

Cooley-Tukey (1965): *Fast Fourier Transform* $\mathcal{O}(N \log N)$ operations

Quantum Fourier Transform

Theorem

There exists a quantum circuit with $\mathcal{O}((\log N)^2)$ gates that computes the QFT.

For $N = 2^n$, we can build such a circuit with $\mathcal{O}(n^2)$

- Hadamard gates $H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$
- controlled phase shifts $R_m = P(\frac{2\pi}{2^m}) = \begin{bmatrix} 1 & 0 \\ 0 & e^{\frac{2\pi i}{2^m}} \end{bmatrix}$
- swaps.

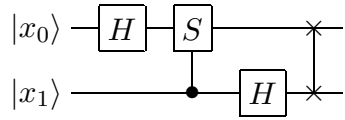
Small values of n

$n = 0$: $\mathcal{F} = I$ ✓

$n = 1$: $\mathcal{F} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} = H$ ✓

$n = 2$: with $S = R_2 = P(\frac{\pi}{2}) = \sqrt{Z}$

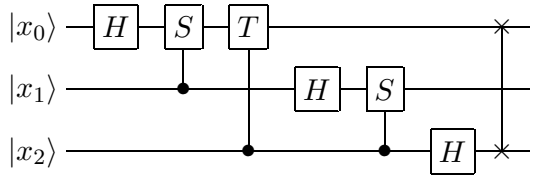
$$\mathcal{F} = \frac{1}{2} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & i & -1 & -i \\ 1 & -1 & 1 & -1 \\ 1 & -i & -1 & i \end{bmatrix} =$$



✓

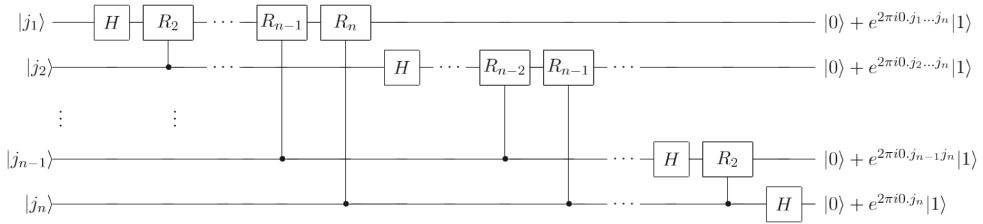
Small values of n

$n = 3$: with $T = R_3 = P(\frac{\pi}{4}) = \sqrt{S}$



Exercise: Verify this in the IBM Composer!

General QFT circuit



- n Hadamard gates
- $1 + 2 + \cdots + (n-1) = \binom{n}{2}$ controlled phase shifts
- $\leq \binom{n}{2}$ swaps

Quantum algorithms

Quantum algorithms

Towards Shor

Period detection

Shor's algorithm

Shor IRL

Grover's algorithm

Grover IRL

Application: Period detection

Suppose $f : \mathbb{Z}_N \to \mathbb{Z}_N$ is r-periodic:

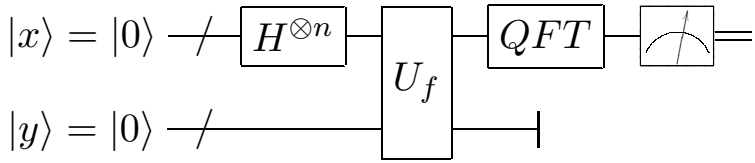
$$f(x + r) = f(x) \qquad \text{for all } x.$$

Problem: find (smallest positive such) r.

A special case of the **hidden subgroup problem**.

Idea: we can detect the period using a Fourier transform.

Quantum period detection



Theorem

If f is r-periodic, then a multiple of $\frac{N}{r}$ is measured.

Remark: for the moment we are assuming that $r \mid N$ (else replace r by $\mathrm{GCD}(r, N)$)

Proof in the $r \mid N$ case

Write $N = rs$.

Evolution of the quantum state:

$$|0\rangle \otimes |0\rangle \overset{H^{\otimes n}}{\mapsto} \frac{1}{\sqrt{N}} \sum_{x<N} |x\rangle \otimes |0\rangle \overset{U_f}{\mapsto} \frac{1}{\sqrt{N}} \sum_{x<N} |x\rangle \otimes |f(x)\rangle$$

$$\overset{\text{QFT}}{\mapsto} \frac{1}{N} \sum_{x<N} \sum_{y<N} \zeta^{xy} |y\rangle \otimes |f(x)\rangle$$

Now write $x = j + kr$, so that $f(x) = f(j)$.

Proof in the $r \mid N$ case

$$|\psi\rangle = \frac{1}{N} \sum_{j<r} \sum_{k<s} \sum_{y<N} \zeta^{(j+kr)y} |y\rangle \otimes |f(j)\rangle = \frac{1}{N} \sum_{y<N} \sum_{j<r} \zeta^{jy} \underbrace{\sum_{k<s} (\zeta^{ry})^k}_{\text{0 unless } \zeta^{ry}=1} |y\rangle \otimes |f(j)\rangle$$

since

$$\sum_{k<s} (\zeta^{ry})^k = \begin{cases} \dfrac{1-(\zeta^{ry})^s}{1-\zeta^{ry}} = \dfrac{1-1}{1-\zeta^{ry}} = 0 & \text{if } \zeta^{ry} \neq 1 \\ \displaystyle\sum_{k<s} 1 = s & \text{if } \zeta^{ry} = 1 \text{ i.e. } s \mid y \end{cases}$$

In the end we have

$$|\psi\rangle = \frac{1}{r} \sum_{t<r} \sum_{j<r} (\zeta^s)^{jt} |ts\rangle \otimes |f(j)\rangle$$

Proof in the $r \mid N$ case

$$|\psi\rangle = \frac{1}{r} \sum_{t<r} \sum_{j<r} (\zeta^s)^{jt} \, |ts\rangle \otimes |f(j)\rangle$$

Only values of y of the form ts have a non-zero probability of being measured:

$$\mathbb{P}\big[\, y = ts \,\big] = \frac{1}{r^2} \left\| \sum_{j<r} (\zeta^s)^{jt} \, |f(j)\rangle \right\|^2$$

In particular, in the special case where all values $f(j)$ are distinct:

$$\mathbb{P}\big[\, y = ts \,\big] = \frac{1}{r^2} \sum_{j<r} |(\zeta^s)^{jt}|^2 = \frac{1}{r^2} \sum_{j<r} 1 = \frac{1}{r}$$

Approximate period detection

Now suppose $f : [\![0, N[\![\to [\![0, N[\![$ is **almost** r**-periodic**:

$$f(x + r) = f(x) \qquad \text{for all } 0 \leq x < N - r.$$

Theorem

If f is almost r-periodic, then an approximate multiple of $\frac{N}{r}$ is probably measured.

And then we can (probably) recover r with classical post-processing.

Analysis in the general case

Write $N = rs + a$ with $0 \leq a < r$. Everything is the same until

$$|\psi\rangle = \frac{1}{N} \sum_{y<N} \sum_{j<r} \zeta^{jy} \sum_{k<s_j} (\zeta^{ry})^k |y\rangle \otimes |f(j)\rangle$$

$$\text{with } s_j = \begin{cases} s+1 & \text{if } 0 \leq j < a \\ s & \text{if } a \leq j < r \end{cases}$$

$$\sum_{k<s_j} (\zeta^{ry})^k = \frac{1-(\zeta^{ry})^{s_j}}{1-\zeta^{ry}} = \zeta^{\frac{ry(s_j-1)}{2}} \sigma_{s_j}\left(y\tfrac{r}{N}\right) \quad \text{where} \quad \sigma_\alpha(x) = \begin{cases} \dfrac{\sin(\alpha\pi x)}{\sin(\pi x)} & x \notin \mathbb{Z} \\ \alpha & x \in \mathbb{Z} \end{cases}$$

Analysis in the general case

$$|\psi\rangle = \frac{1}{N} \sum_{y<N} \sum_{j<r} \zeta^{jy + \frac{ry(s_j - 1)}{2}} \sigma_{s_j}\left(y \tfrac{r}{N}\right) |y\rangle \otimes |f(j)\rangle$$

To simplify: under the assumption that all the p values $f(j)$ are distinct:

$$\mathbb{P}[\,y\,] = \frac{1}{N^2} \left(a\, \sigma_{s+1}\left(y \tfrac{r}{N}\right)^2 + (r - a)\, \sigma_s\left(y \tfrac{r}{N}\right)^2 \right)$$

cf. Sage visualization

Analysis in the general case

Proposition

The probability that $\lfloor t\frac{N}{r} \rfloor$ or $\lceil t\frac{N}{r} \rceil$ is measured is asymptotically $\geq \frac{4}{\pi^2} \approx 40\%$

with t uniformly distributed among $[\![0, r[\![$.

Thus probability $.4\left(1 - \frac{1}{r}\right)$ of getting a "good" result y.

Fact: If $N > 2r^2$, the period r can then be efficiently recovered since $\frac{t}{r}$ appears in lowest terms in the continued fraction expansion of $\frac{y}{N}$ and if r is large, t and r will most likely be coprime (probability $\approx \frac{1}{\log \log r}$ of failure).

Quantum algorithms

Quantum algorithms

Towards Shor

Period detection

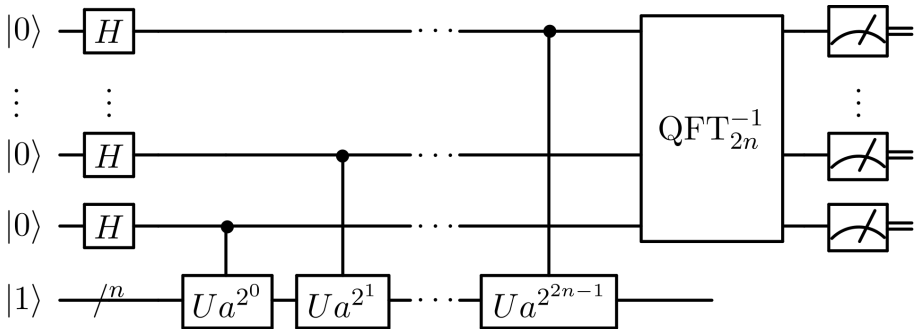
Shor's algorithm

Shor IRL

Grover's algorithm

Grover IRL

Shor's algorithm (1994)



Shor's algorithm

(Probably) factors an integer N with quantum complexity

$$\mathcal{O}\big((\log N)^2(\log\log N)(\log\log\log N)\big)$$

This is much better than the best currently known classical algorithm

that has complexity

$$\mathcal{O}\big(e^{1.9(\log N)^{\frac{1}{3}}(\log\log N)^{\frac{2}{3}}}\big)$$

A note on complexity

If N is an integer that can be written on n bits:

$$N < 2^n \qquad \text{so} \qquad \log_2(N) < n.$$

This is the natural parametrer to measure the size of an integer.

Factorization on a classical computer: $\mathcal{O}\left(e^{1.9 n^{\frac{1}{3}} (\log n)^{\frac{2}{3}}}\right)$

Factorization on a quantum computer: $\mathcal{O}\left(n^2 (\log n)(\log\log n)\right)$ or even $\mathcal{O}\left(n^2 (\log n)\right)$

quasiexponential speedup

Factoring vs period finding

Suppose $N = pq$ with p and q two distinct prime numbers.

Theorem (Euler)

For any integer a coprime with N, we have $a^r \equiv 1 \bmod N$ for

$$r = \varphi(N) = (p-1)(q-1).$$

In other words: r is a period for the function $f : x \mapsto a^x \bmod N$.

Reduction from factoring to period finding

To factor $N = pq$:

- pick a random integer $a \in [\![0, N[\![$
- with high probability $\mathrm{GCD}(a, N) = 1$
- if $x \mapsto a^x \bmod N$ has *odd* period r, pick another a
- so now we have an integer a of even multiplicative order $r = 2s$: $a^{2s} \equiv 1 \bmod N$

$$N \mid a^{2s} - 1 = (a^s - 1)(a^s + 1)$$

- there is a 50% chance that $\mathrm{GCD}(N, a^s \pm 1)$ are non-trivial divisors of N

Small example: $N = 21$

Try $a = 4$:

$$4 \quad \to \quad 4^2 = 16 \quad \to \quad 4^3 \equiv 1 \qquad \text{✗}$$

Try $a = 5$:

$$5 \quad \to \quad 5^2 \equiv 4 \quad \to \quad 5^3 \equiv 20 \quad \to \quad 5^4 \equiv 16 \quad \to \quad 5^5 \equiv 17 \quad \to \quad 5^6 \equiv 1$$

but $\text{GCD}(5^3 - 1, 21) = 1$, $\text{GCD}(5^3 + 1, 21) = 21$ ✗

Try $a = 2$:

$$2 \quad \to \quad 2^2 = 4 \quad \to \quad 2^3 = 8 \quad \to \quad 2^4 = 16 \quad \to \quad 2^5 \equiv 11 \quad \to \quad 2^6 \equiv 1$$

and $\text{GCD}(2^3 - 1, 21) = 7$, $\text{GCD}(2^3 + 1, 21) = 3$ ✓

Implementation

So we need a quantum implementation U_f of the function

$$f(x) = a^x \bmod N.$$

Shor picks $Q = 2^n > N^2$ in order to be able to apply postprocessing and considers the function $f(x)$ for $x \in [\![0, Q[\![$.

If x is written in binary:

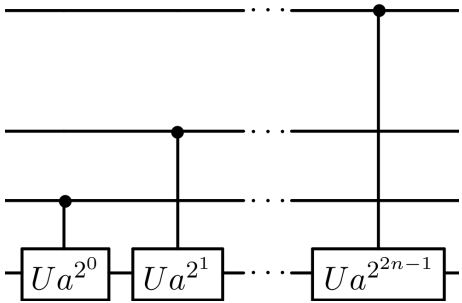
$$x = 2^{n-1} b_{n-1} + \cdots + 2^1 b_1 + 2^0 b_0$$

then

$$a^x = (a^{2^{n-1}})^{b_{n-1}} \cdots (a^{2^1})^{b_1} \cdot (a^{2^0})^{b_0}$$

Implementation

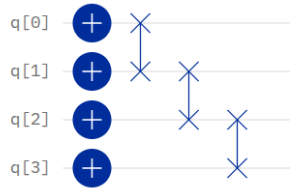
Thus we only need to implement "multiplication by a^{2^k} mod N"



This is actually the difficult part! One approach is to translate *reversible* classical arithmetical circuits into quantum circuits + classical pre-processing.

Exercise

Verify that the following circuit computes $x \mapsto 7x \mod 15$:



Can you come up similarly with circuits that multiply by 7^2, 7^4, 7^8, 7^16 ?

(In order to implement period finding we would need controlled versions of these circuits, then apply period finding)

Quantum factoring: summary

To find a nontrivial factor of N:

- Pick $Q = 2^n$ large enough
- Choose a coprime with N randomly
- Implement modular exponentiation $f(x) = a^x \bmod N$ as a quantum circuit
- Apply QFT + classical post-processing to recover period r of f
- Repeat until $r = 2s$ is even and $\mathrm{GCD}(a^s - 1, N) \neq 1, N$
- Output $\mathrm{GCD}(a^s - 1, N)$

Quantum algorithms

Quantum algorithms

Towards Shor

Period detection

Shor's algorithm

Shor IRL

Grover's algorithm

Grover IRL

Recent history of factoring

- 1977: RSA public-key cryptosystem, based on the difficulty of factoring large quasiprime integers $N = pq$

  Scientific American Challenge: factor RSA-129 $\approx 2^{426}$

- 1981: Quadratic sieve
- 1994: RSA-129 factored (1600 computers)
- 1996: General number field sieve, RSA-130 $\approx 2^{430}$ factored

  $\vdots$

- Feb. 28, 2020: RSA-250 $\approx 2^{829}$ factored (P. Zimmerman, INRIA)

  2048-bit RSA moduli still considered out of reach for the next 25 years

Quantum computers might change that

Computing

How a quantum computer could break 2048-bit RSA encryption in 8 hours

A new study shows that quantum technology will catch up with today's encryption standards much sooner than expected. That should worry anybody who needs to store data securely for 25 years or so.

by **Emerging Technology from the arXiv** May 30, 2019

Quantum computers might change that

IBM unveils its first commercial quantum computer

Frederic Lardinois @fredericl / 5:29 pm CET • January 8, 2019

Comment



At CES, **IBM** today announced its first commercial quantum computer for use outside of the lab. The 20-qubit system combines into a single package the quantum and classical computing parts it takes to use a machine like this for research and business applications. That package, the IBM Q system, is still huge, of course, but it includes everything a company would need to get started with its quantum computing experiments, including all

How to factor 2048-bit RSA integers in 8 hours

Quantum Physics

How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits

Craig Gidney, Martin Ekerå

(Submitted on 23 May 2019 (v1), last revised 5 Dec 2019 (this version, v2))

We significantly reduce the cost of factoring integers and computing discrete logarithms in finite fields on a quantum computer by combining techniques from Shor 1994, Griffiths-Niu 1996, Zalka 2006, Fowler 2012, Ekerå-Håstad 2017, Ekerå 2017, Ekerå 2018, Gidney-Fowler 2019, Gidney 2019. We estimate the approximate cost of our construction using plausible physical assumptions for large-scale superconducting qubit platforms: a planar grid of qubits with nearest-neighbor connectivity, a characteristic physical gate error rate of 10^{-3}, a surface code cycle time of 1 microsecond, and a reaction time of 10 microseconds. We account for factors that are normally ignored such as noise, the need to make repeated attempts, and the spacetime layout of the computation. When factoring 2048 bit RSA integers, our construction's spacetime volume is a hundredfold less than comparable estimates from earlier works (Fowler et al. 2012, Gheorghiu et al. 2019). In the abstract circuit model (which ignores overheads from distillation, routing, and error correction) our construction uses $3n + 0.002n \lg n$ logical qubits, $0.3n^3 + 0.0005n^3 \lg n$ Toffolis, and $500n^2 + n^2 \lg n$ measurement depth to factor n-bit RSA integers. We quantify the cryptographic implications of our work, both for RSA and for schemes based on the DLP in finite fields.

Comments: 26 pages, 10 figures, 5 tables
Subjects: **Quantum Physics (quant-ph)**
Cite as: arXiv:1905.09749 [quant-ph]

Steady improvement

Quantum Physics

[Submitted on 21 May 2025]

How to factor 2048 bit RSA integers with less than a million noisy qubits

Craig Gidney

Planning the transition to quantum-safe cryptosystems requires understanding the cost of quantum attacks on vulnerable cryptosystems. In Gidney+Ekerå 2019, I co-published an estimate stating that 2048 bit RSA integers could be factored in eight hours by a quantum computer with 20 million noisy qubits. In this paper, I substantially reduce the number of qubits required. I estimate that a 2048 bit RSA integer could be factored in less than a week by a quantum computer with less than a million noisy qubits. I make the same assumptions as in 2019: a square grid of qubits with nearest neighbor connections, a uniform gate error rate of 0.1%, a surface code cycle time of 1 microsecond, and a control system reaction time of 10 microseconds.

The qubit count reduction comes mainly from using approximate residue arithmetic (Chevignard+Fouque+Schrottenloher 2024), from storing idle logical qubits with yoked surface codes (Gidney+Newman+Brooks+Jones 2023), and from allocating less space to magic state distillation by using magic state cultivation (Gidney+Shutty+Jones 2024). The longer runtime is mainly due to performing more Toffoli gates and using fewer magic state factories compared to Gidney+Ekerå 2019. That said, I reduce the Toffoli count by over 100x compared to Chevignard+Fouque+Schrottenloher 2024.

Subjects: **Quantum Physics (quant-ph)**

Back of the envelope estimation

Today: ~ 1200 noisy qubits

When would we have functional quantum computers with 10^6 qubits ?

Multiplicative factor of $833.3 \approx 2^{9.7}$

9.7 doublings – assuming the continuing validity of Rose's law

$$\implies 9.7 \times 1.5 \approx 15 \text{ years of safety}$$

This is starting to be a bit worrisome for the security of long-term secrets. (Current recommendation is to use 3096-bit moduli with RSA anyway)

The uncertain future of modular arithmetic-based cryptography

Large-scale quantum computers would have a definite asymptotic quantum advantage over classical algorithms. Indeed, for:

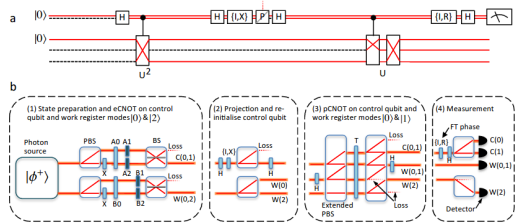
- RSA encryption and signatures (hardness of factoring)
- DSA signatures,
- elliptic curve cryptography (hardess of the *discrete logarithm problem*)

a quantum attacker breaks the system essentially as fast as classical users Alice and Bob users are able to use it with the appropriate private key ...

$\Longrightarrow$ ongoing NIST Post-quantum cryptography standardization process

Quantum factoring records

- 2001: 15 factored (IBM, Shor on 7 qubits)
- 2012: 21 factored ($a = 4$, 1 qubit + 1 qutrit)



"Compiled version of Shor's algorithm"

(see Pretending to factor large numbers on a quantum computer)

Quantum algorithms

Quantum algorithms

Towards Shor

Period detection

Shor's algorithm

Shor IRL

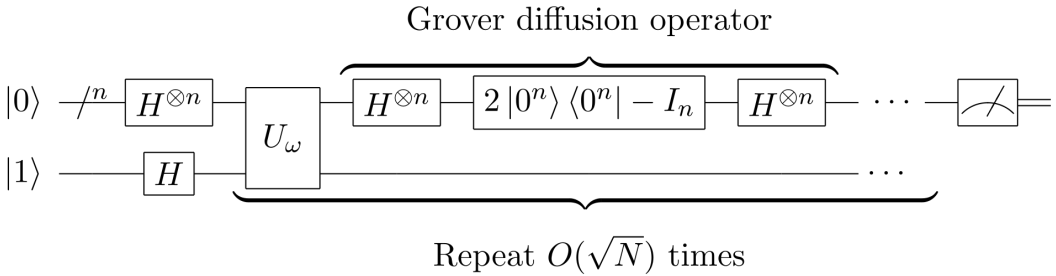
Grover's algorithm

Grover IRL

Grover (1970)



Grover (1996)


Grover diffusion operator
$|0\rangle$
$|1\rangle$
$/^n$
$H^{\otimes n}$
H
U_ω
$H^{\otimes n}$
$2\,|0^n\rangle\,\langle 0^n| - I_n$
$H^{\otimes n}$
$\cdots$
$\cdots$
Repeat $O(\sqrt{N})$ times

Search problem

Suppose we have a decision function $f : X \to \{0, 1\}$ defined on a set X of size N.

The search problem defined by f is to find some $x \in X$ for which $f(x) = 1$.

Examples: database queries, factoring integers, bitcoin mining, ...

In the general (unstructured) case: a classical algorithm requires $\mathcal{O}(N)$ queries.

(Of course can do better if *e.g.* the data is sorted)

Grover's algorithm

Performs unstructured searches for arbitrary criteria in $\mathcal{O}(\sqrt{N})$ time.

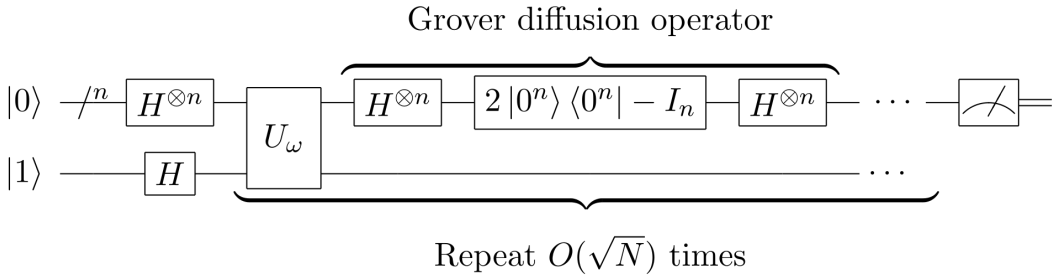
$\Longrightarrow$ **quadratic speedup**

Works in two steps:

- phase inversion
- amplitude amplification

iterated a certain number of times

Circuit for Grover's algorithm



Phase inversion

Simplifying assumptions:

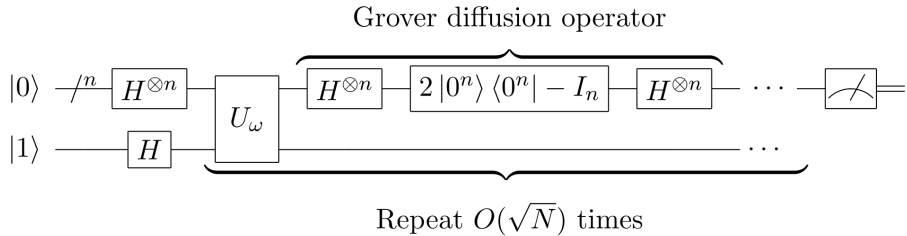
- $X = [\![0, N[\![$
- $N = 2^n$
- the equation $f(x) = 1$ admits a unique solution $\omega \in X$

So the problem is now: find $\omega \in X$ given access to an oracle for $f : [\![0, N[\![\to \{0, 1\}$

$$\text{where} \quad f(x) = \begin{cases} 1 & \text{if } x = \omega \\ 0 & \text{else.} \end{cases}$$

Phase inversion

ω is detected by inverting its phase: "$U_\omega|x\rangle = (-1)^{f(x)}|x\rangle$"



Actually

$$U_\omega\,|x\rangle \otimes |-\rangle = (-1)^{f(x)}\,|x\rangle \otimes |-\rangle$$

This is exactly what the oracle U_f does! So in fact "$U_\omega = U_f$".

Amplitude amplification

The **Grover diffusion operator** G is

$$G = 2|s\rangle\langle s| - I$$

where

$$|s\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle.$$

Geometrical interpretation:

$$G|s\rangle = |s\rangle$$

$$G|\psi\rangle = -|\psi\rangle \qquad \text{when } \langle s|\psi\rangle = 0$$

$U_s = -G$ is a reflection through the hyperplane normal to $|s\rangle$

Amplitude amplification

Remark: U_ω is a reflection, too.

Actually U_ω acts on $\mathcal{V} = \mathcal{V}_N \otimes |-\rangle$ as

$$I - 2|\omega\rangle\langle\omega| = \mathrm{diag}(1, \ldots, \underbrace{-1}_{\omega}, \ldots, 1).$$

In general $I - 2|\psi\rangle\langle\psi|$ is a reflection through the hyperplane normal to $|\psi\rangle$.

GU_ω: unitary transformation of $\mathcal{V}$ that inverts every vector $|\psi\rangle$ orthogonal to both $|s\rangle$ and $|\omega\rangle$ – and acts as a rotation in the plan spanned by $|s\rangle$ and $|\omega\rangle$

Amplitude amplification

Consider unitary $|s'\rangle \sim |s\rangle - \langle\omega|s\rangle|\omega\rangle$, and write $\langle\omega|s\rangle = \frac{1}{\sqrt{N}} = \sin\frac{\theta}{2}$.

Initial state:

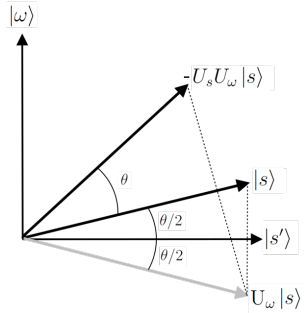
$$|\psi\rangle = |s\rangle = \cos\tfrac{\theta}{2}\,|s'\rangle + \sin\tfrac{\theta}{2}\,|\omega\rangle$$

GU_ω is a rotation of θ (exercise!), so after k iterations:

$$(GU_\omega)^k|\psi\rangle = \cos(\tfrac{\theta}{2} + k\theta)\,|s'\rangle + \sin(\tfrac{\theta}{2} + k\theta)\,|\omega\rangle$$

$$\mathbb{P}\big[\,\mathcal{M}(GU_\omega)^k|\psi\rangle = |\omega\rangle\,\big] = \sin^2(\tfrac{\theta}{2} + k\theta)$$

Optimal number of iterations



Each iteration brings the state closer to $|\omega\rangle$ by an angle of $\theta = 2 \arcsin \frac{1}{\sqrt{N}}$.

Until it starts moving away... Sage visualization

Optimal number of iterations

So, in order to maximize the probability of measuring $|\omega\rangle$, take

$$(k + \tfrac{1}{2})\theta \approx \frac{\pi}{2} \qquad \Longleftrightarrow \qquad k \approx \frac{\pi}{2\theta} - \frac{1}{2}$$

When N is large (interesting case!) we have $\theta \approx \sin\theta = \dfrac{2}{\sqrt{N}}$

so the optimal number of iterations is $k \approx \dfrac{\pi\sqrt{N}}{4}$.

Closely related to this rather surprising way to approximate π !

Implementation of G

$$G = 2|s\rangle\langle s| - I$$

- G is more easily computed if we change the basis:

$$G = H^{\otimes n} \otimes \underbrace{(2|0\rangle\langle 0| - I)}_{G_0} \otimes H^{\otimes n}$$

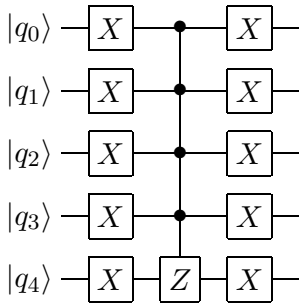
- $G_0 \sim -G_0 = U_0 = \mathrm{diag}(-1, 1, \ldots, 1)$:

$$U_0|x\rangle = \begin{cases} -|x\rangle & \text{if } x = 0 \\ |x\rangle & \text{if } x \neq 0. \end{cases}$$

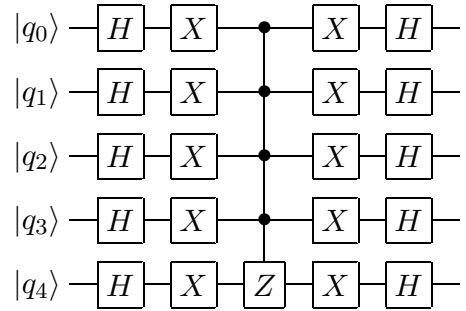
Implementation of G

Example: with $n = 5$

G_0:



G:



Quantum algorithms

Quantum algorithms

Towards Shor

Period detection

Shor's algorithm

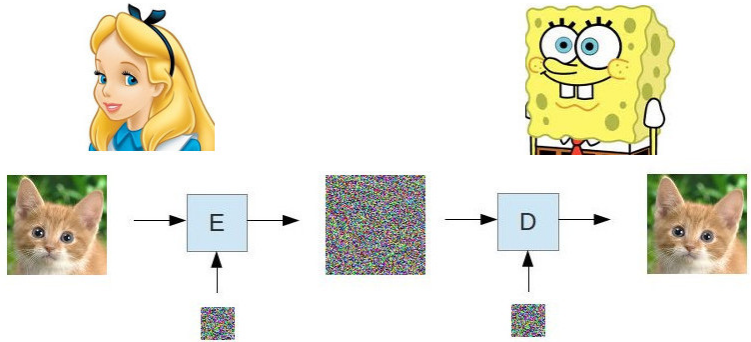
Shor IRL

Grover's algorithm

Grover IRL

Application: breaking cryptography

Alice sends secret messages to Bob:



Application: breaking cryptography

They agree on a secret n-bit encryption key k.

Alice encrypts her messages with k:

$$c = E(k, m)$$

and Bob decrypts them using the same k:

$$m = D(k, c).$$

Known plaintext attack

Imagine the attacker, Eve, knows the message m corresponding to *one* ciphertext c.

Then she can try to recover the secret key k in order to be able to decrypt *all* messages exchanged by Alice and Bob.

(A plausible scenario: this is exactly what happened with Enigma during WWII).

This is a search problem: she's looking for $k \in [\![0, 2^n[\![$ for which $D(k, c) = m$.

Brute force attack on the key

Suppose $n = 128$ (today's standard for AES).

With a classical computer: Eve will need to go through the 2^{128} possibilities: impractical for at least the next 30 years.

$\Longrightarrow$ secure communication ✓

With a quantum computer running Grover's algorithm: Eve will recover the key in $\sqrt{2^{128}} = 2^{64}$ steps: doable today using specialized hardware.

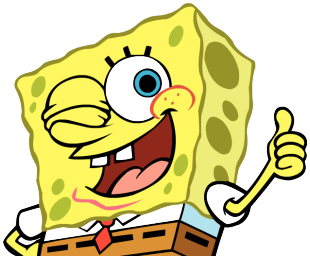
$\Longrightarrow$ no more confidentiality ✗

Solution

In case this happens:

"post-quantum symmetric cryptography": just move to 256-bit keys

No biggie!



Quantum search

Recall: Grover's algorithm performs unstructured search with $\mathcal{O}(\sqrt{N})$ complexity.

Often described as *efficient database lookup*.

Catch: needs access to an oracle for the selection function $f \ldots$

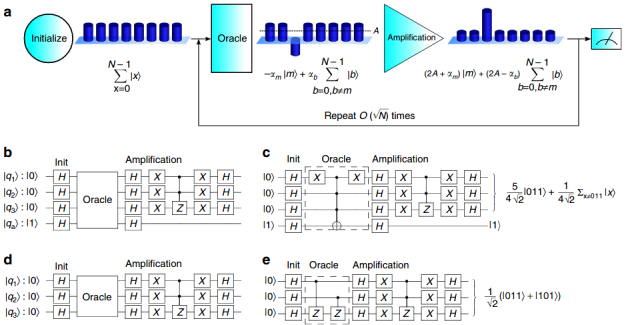
For real-life applications: U_f needs to be implemented:

- either as an actual computation on keys *e.g.* AES in Q# (2019), or
- by loading a value in quantum memory and performing a comparision

Most implementations use "toy selection functions" and should be considered proofs of concepts (or physical experiments) more than actual improvements over classical search

Example

Complete 3-qubit Grover search on a programmable quantum computer (Nature 2017)



(scalable trapped-ion architecture)

Grover's algorithm

Given a generic quantum computable function $x \mapsto f(x)$, yields preimages of f for chosen values of y with $\mathcal{O}(\sqrt{N})$ quantum evaluations of f

$$f(x) = y \iff x \in f^{-1}(y)$$

Think of it as a "quantum preimage finder" or "quantum function inverter"

